

Softwaregestützte Modellbildung und Simulation: Exercise 2

Principal Component Analysis (PCA)

Peleg Bar Sapir

June 4, 2026

Principal Component Analysis (PCA) is an application of Singular Value Decomposition (SVD) ...

The data set is ...

1. Load the data into a single NumPy array, using `numpy.loadtxt()`. Mathematically, we will call this data array **X**. Note that the **rows** of **X** are the separate *samples* (measurements), while its **columns** are the variables (features): *sepal length*, *sepal width*, *petal length* and *petal width*.
2. Center and normalize the data by subtracting from each **column** of **x** its mean, then dividing it by its standard deviation. Use the functions `numpy.mean()` and `numpy.std()`, respectively. Which axis should you use for these operations? Call the resulting matrix **B**.
3. Apply the SVD method to the matrix **B**:

```
1 U, S, VT = numpy.linalg.SVD(B, full_matrices=False)
```

(where **U**, **S**, **VT** are, respectively, the matrices **U**, **Σ**, and **V[†]** in the SVD of **B**)

4. In the terms of PCA, the columns of **U** are called the *Principal Components* (PC) of **B**. The number of principal components *N* is the number of variables in the data:

$$PC_1, PC_2, \dots, PC_N.$$

The corresponding Singular Values are the diagonal elements of the matrix **Σ**, denoted as

$$\sigma_1, \sigma_2, \dots, \sigma_N.$$

The singular values are ordered by absolute value in descending order. The variance in PC_i is given by

$$\text{Var}_i = \frac{\sigma_i^2}{r-1},$$

where r is the number of *samples* and $i = 1, 2, \dots, N$.

Plot the variance for each PC in descending order.

5. What do we get if we divide each Var_i by the sum of all variance terms? Plot these values as function of i . (**hint**: what is the sum of these terms?)
6. Plot the *cumulative sum* of the values calculated in the previous step, again against i (use the function `numpy.cumsum()`). Does the cumulative sum seem to converge, and if so - to what limit? Explain briefly.
7. Since the PCs are ordered by their variance in descending order, we can choose the first two PCs, PC_1 and PC_2 , and project the data on to them. This will keep the most variance in the data while allowing us to visualize it. The matrix $\mathbf{U}$ has the PCs as its columns, and $\mathbf{\Sigma}$ are their respective values - both are sorted by variance in descending order. Therefore, the full data in terms of the PCs is given by

$$\mathbf{P} = \mathbf{U} \cdot \mathbf{\Sigma},$$

and therefore, the projection of the data on the first k PCs is given by the matrix product of the first k rows of $\mathbf{U}$ and the first k rows of the matrix $\mathbf{\Sigma}$:

$$\mathbf{P}_{j=1}^k = \mathbf{U}_{j=1}^k \cdot \mathbf{\Sigma}_{j=1}^k.$$

Implement this last projection on the first two PCs (i.e. $k = 2$), and plot the result as a scatter plot of PC_2 vs. PC_1 . Use `axes.scatter(x, y)`, where $\mathbf{x}$ and $\mathbf{y}$ are the components of the data points in the PC_1 and PC_2 basis vectors, respectively.